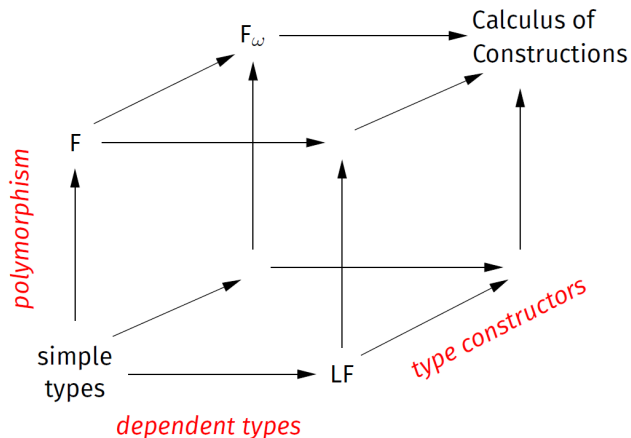


System F_ω : Higher-Order Polymorphism

Reference: Chapters 29 and 30 in Pierce's *TAPL*

Lambda Cube



This class: System F_ω .

Recall System F

(Terms) $M ::= x \mid \lambda x : \tau. M \mid M M \mid \Lambda \alpha. M \mid M \langle \tau \rangle$

(Types) $\tau ::= \alpha \mid \mathbf{T} \mid \tau \rightarrow \tau \mid \forall \alpha. \tau$

(Values) $v ::= \lambda x : \tau. M \mid \Lambda \alpha. M$

(Contexts) $\Gamma ::= \bullet \mid \Gamma, x : \tau$

(TypeVarContexts) $\Delta ::= \bullet \mid \Delta, \alpha$

Reduction

$$\frac{}{(\lambda x : \tau. M_1) M_2 \longrightarrow M_1[M_2/x]} \text{ (E-APPABS)}$$
$$\frac{}{(\Lambda \alpha. M_1) \langle \tau_2 \rangle \longrightarrow M_1[\tau_2/\alpha]} \text{ (E-TAPPTABS)}$$

Recall System F

Type well-formedness: $\Delta \vdash \tau$

Typing judgment: $\Delta; \Gamma \vdash M : \tau$

$$\frac{}{\Delta; \Gamma, x : \tau \vdash x : \tau} \text{ (T-VAR)} \qquad \frac{\Delta \vdash \tau_1 \quad \Delta; \Gamma, x : \tau_1 \vdash M : \tau_2}{\Delta; \Gamma \vdash (\lambda x : \tau_1. M) : \tau_1 \rightarrow \tau_2} \text{ (T-ABS)}$$

$$\frac{\Delta; \Gamma \vdash M_1 : \tau \rightarrow \tau' \quad \Delta; \Gamma \vdash M_2 : \tau}{\Delta; \Gamma \vdash M_1 M_2 : \tau'} \text{ (T-APP)}$$

$$\frac{\Delta, \alpha; \Gamma \vdash M : \tau}{\Delta; \Gamma \vdash (\lambda \alpha. M) : \forall \alpha. \tau} \text{ (T-TABS)} \qquad \frac{\Delta; \Gamma \vdash M_1 : \forall \alpha. \tau \quad \Delta \vdash \tau_2}{\Delta; \Gamma \vdash M_1 \langle \tau_2 \rangle : \tau[\tau_2/\alpha]} \text{ (T-TAPP)}$$

Recall System F

We can encode boolean values:

$$\begin{aligned}\text{True} &\stackrel{\text{def}}{=} \Lambda\alpha. \lambda x : \alpha. \lambda y : \alpha. x \\ \text{False} &\stackrel{\text{def}}{=} \Lambda\alpha. \lambda x : \alpha. \lambda y : \alpha. y\end{aligned}$$

Their type: $\text{Bool} \stackrel{\text{def}}{=} \forall\alpha. \alpha \rightarrow \alpha \rightarrow \alpha.$

Recall System F

We can encode boolean values:

$$\begin{aligned}\text{True} &\stackrel{\text{def}}{=} \Lambda\alpha. \lambda x : \alpha. \lambda y : \alpha. x \\ \text{False} &\stackrel{\text{def}}{=} \Lambda\alpha. \lambda x : \alpha. \lambda y : \alpha. y\end{aligned}$$

Their type: $\text{Bool} \stackrel{\text{def}}{=} \forall\alpha. \alpha \rightarrow \alpha \rightarrow \alpha.$

We can also encode a pair (x, y) , where x has type T_1 and y has type T_2 :

$$(x, y) \stackrel{\text{def}}{=} \Lambda\alpha. \lambda f : T_1 \rightarrow T_2 \rightarrow \alpha. f\ x\ y$$

Its type:

$$\text{Pair } T_1\ T_2 \stackrel{\text{def}}{=} \forall\alpha. (T_1 \rightarrow T_2 \rightarrow \alpha) \rightarrow \alpha$$

Type Operators (a.k.a. Type Constructors)

Type operators: type-level abstraction.

“Types depend on types.”

Example

$$\text{Pair} \stackrel{\text{def}}{=} \lambda\alpha_1. \lambda\alpha_2. \forall\alpha. (\alpha_1 \rightarrow \alpha_2 \rightarrow \alpha) \rightarrow \alpha$$

Kinds

The abstraction and application mechanism at the *type* level gives us the ability to write meaningless type expressions. For example, (Bool Nat) makes no more sense than $(\text{True } 6)$ at the term level.

To prevent this sort of nonsense, we introduce a system of **kinds** that classify type expressions.

$$(\text{Kinds}) \quad \kappa ::= * \mid \kappa \rightarrow \kappa$$

Kinds are “the types of types”. Examples:

type	Bool	$\text{Bool} \rightarrow \text{Bool}$	Pair
kind	*	*	$* \rightarrow * \rightarrow *$

Example: Pair

$$\text{Pair } T_1 \ T_2 \stackrel{\text{def}}{=} \forall \alpha. (T_1 \rightarrow T_2 \rightarrow \alpha) \rightarrow \alpha$$

$$(x, y) \stackrel{\text{def}}{=} \Lambda \alpha. \lambda f : T_1 \rightarrow T_2 \rightarrow \alpha. f \ x \ y$$

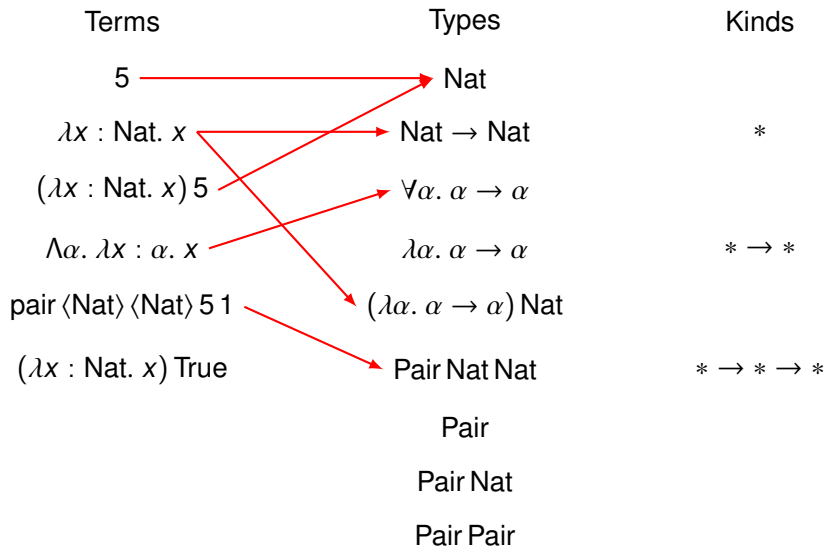
$$\text{Pair} \stackrel{\text{def}}{=} \lambda \alpha_1 :: *. \lambda \alpha_2 :: *. \forall \alpha. (\alpha_1 \rightarrow \alpha_2 \rightarrow \alpha) \rightarrow \alpha$$

$$\begin{aligned} \text{pair} &\stackrel{\text{def}}{=} \Lambda \alpha_1. \Lambda \alpha_2. \lambda x : \alpha_1. \lambda y : \alpha_2. \Lambda \alpha. \lambda f : \alpha_1 \rightarrow \alpha_2 \rightarrow \alpha. f \ x \ y \\ (x, y) &\stackrel{\text{def}}{=} \text{pair } \langle T_1 \rangle \langle T_2 \rangle \ x \ y \end{aligned}$$

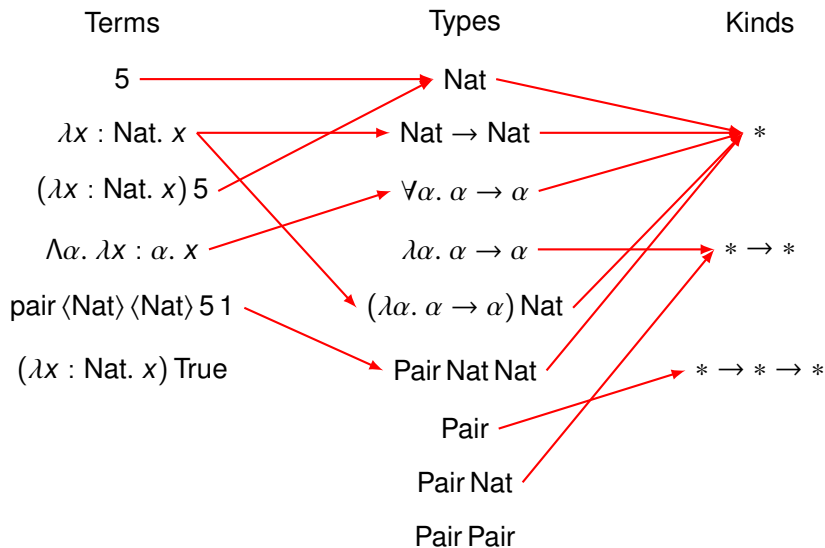
Terms, Types and Kinds

Terms	Types	Kinds
5	Nat	
$\lambda x : \text{Nat}. x$	$\text{Nat} \rightarrow \text{Nat}$	*
$(\lambda x : \text{Nat}. x) 5$	$\forall \alpha. \alpha \rightarrow \alpha$	
$\Lambda \alpha. \lambda x : \alpha. x$	$\lambda \alpha. \alpha \rightarrow \alpha$	$* \rightarrow *$
$\text{pair } \langle \text{Nat} \rangle \langle \text{Nat} \rangle 5 1$	$(\lambda \alpha. \alpha \rightarrow \alpha) \text{Nat}$	
$(\lambda x : \text{Nat}. x) \text{True}$	Pair Nat Nat	$* \rightarrow * \rightarrow *$
	Pair	
	Pair Nat	
	Pair Pair	

Terms, Types and Kinds



Terms, Types and Kinds



Discussions

Why stop at three levels of expressions? Couldn't we go on to add a fourth level to classify “kinds”, and so continue without end?

Such systems have been investigated by the *pure type systems* community.

However, for programming languages, three levels have proved sufficient. In essentially all statically typed programming languages, it is relatively rare for language designers to offer programmers even the full power of the type operators. Some languages (e.g., Java) offer only a few built-in type operators like `Array`, with no facilities for defining new ones.

$\lambda\omega$: Simply-Typed λ -Calculus with Type Operators

(Terms)	M	$::=$	$x \mid \lambda x : \tau. M \mid M M$
(Types)	τ	$::=$	$\alpha \mid \mathbf{T} \mid \tau \rightarrow \tau \mid \lambda \alpha :: \kappa. \tau \mid \tau \tau$
(Kinds)	κ	$::=$	$* \mid \kappa \rightarrow \kappa$
(Values)	v	$::=$	$\lambda x : \tau. M$

- ▶ Type variable α
- ▶ Operator abstraction $\lambda \alpha :: \kappa. \tau$
- ▶ Operator application $\tau \tau$
- ▶ Kind of proper types $*$
- ▶ Kind of operators $\kappa \rightarrow \kappa$

Reduction rules are the same as STLC.

$\lambda\omega$ – Statics

(Terms) $M ::= x \mid \lambda x : \tau. M \mid M M$

(Types) $\tau ::= \alpha \mid \mathbf{T} \mid \tau \rightarrow \tau \mid \lambda \alpha :: \kappa. \tau \mid \tau \tau$

(Kinds) $\kappa ::= * \mid \kappa \rightarrow \kappa$

(Values) $v ::= \lambda x : \tau. M$

(Contexts) $\Gamma ::= \bullet \mid \Gamma, x : \tau$

(TypeVarContexts) $\Delta ::= \bullet \mid \Delta, \alpha :: \kappa$

Kinding judgment: $\Delta \vdash \tau :: \kappa$

Type equivalence: $\tau \equiv \tau'$

Typing judgment: $\Delta; \Gamma \vdash M : \tau$

$\lambda\omega$ – Statics – Kinding

$$\begin{array}{c} \frac{}{\Delta, \alpha :: \kappa \vdash \alpha :: \kappa} \quad \frac{}{\Delta \vdash \mathbf{T} :: *} \quad \frac{\Delta \vdash \tau_1 :: * \quad \Delta \vdash \tau_2 :: *}{\Delta \vdash \tau_1 \rightarrow \tau_2 :: *} \\[2ex] \frac{\Delta, \alpha :: \kappa \vdash \tau :: \kappa'}{\Delta \vdash \lambda \alpha :: \kappa. \tau :: \kappa \rightarrow \kappa'} \quad \frac{\Delta \vdash \tau_1 :: \kappa \rightarrow \kappa' \quad \Delta \vdash \tau_2 :: \kappa}{\Delta \vdash \tau_1 \tau_2 :: \kappa'} \end{array}$$

$\lambda\omega$ – Statics – Type Equivalence

$$\overline{(\lambda\alpha :: \kappa. \tau_1) \tau_2 \equiv \tau_1[\tau_2/\alpha]}$$

$$\frac{\tau_1 \equiv \tau'_1 \quad \tau_2 \equiv \tau'_2}{\tau_1 \rightarrow \tau_2 \equiv \tau'_1 \rightarrow \tau'_2}$$

$$\frac{\tau \equiv \tau'}{\lambda\alpha :: \kappa. \tau \equiv \lambda\alpha :: \kappa. \tau'} \qquad \frac{\tau_1 \equiv \tau'_1 \quad \tau_2 \equiv \tau'_2}{\tau_1 \tau_2 \equiv \tau'_1 \tau'_2}$$

$$\frac{}{\tau \equiv \tau} \qquad \frac{\tau \equiv \tau'}{\tau' \equiv \tau} \qquad \frac{\tau \equiv \tau' \quad \tau \equiv \tau''}{\tau \equiv \tau''}$$

$\lambda\omega$ – Statics – Typing

$$\frac{}{\Delta; \Gamma, X : \tau \vdash X : \tau} \text{ (T-VAR)}$$

$$\frac{\Delta \vdash \tau_1 :: * \quad \Delta; \Gamma, X : \tau_1 \vdash M : \tau_2}{\Delta; \Gamma \vdash (\lambda X : \tau_1. M) : \tau_1 \rightarrow \tau_2} \text{ (T-ABS)}$$

$$\frac{\Delta; \Gamma \vdash M_1 : \tau \rightarrow \tau' \quad \Delta; \Gamma \vdash M_2 : \tau}{\Delta; \Gamma \vdash M_1 M_2 : \tau'} \text{ (T-APP)}$$

$$\frac{\Delta; \Gamma \vdash M : \tau \quad \tau \equiv \tau' \quad \Gamma \vdash \tau' :: *}{\Delta; \Gamma \vdash M : \tau'} \text{ (T-EQ)}$$

F_ω : Combining System F and $\lambda\underline{\omega}$

(Terms) $M ::= x \mid \lambda x : \tau. M \mid M M \mid \Lambda \alpha :: \kappa. M \mid M \langle \tau \rangle$

(Types) $\tau ::= \alpha \mid \mathbf{T} \mid \tau \rightarrow \tau \mid \forall \alpha :: \kappa. \tau \mid \lambda \alpha :: \kappa. \tau \mid \tau \tau$

(Kinds) $\kappa ::= * \mid \kappa \rightarrow \kappa$

(Values) $v ::= \lambda x : \tau. M \mid \Lambda \alpha :: \kappa. M$

(Ctxts) $\Gamma ::= \bullet \mid \Gamma, x : \tau$

(TVCtxts) $\Delta ::= \bullet \mid \Delta, \alpha :: \kappa$

F_ω – Reduction

$$\frac{}{(\lambda x : \tau. M_1) M_2 \longrightarrow M_1[M_2/x]} \text{ (E-APPABS)}$$

$$\frac{M_1 \longrightarrow M'_1}{M_1 M_2 \longrightarrow M'_1 M_2} \text{ (E-APP1)}$$

$$\frac{M_2 \longrightarrow M'_2}{M_1 M_2 \longrightarrow M'_1 M'_2} \text{ (E-APP2)}$$

$$\frac{M \longrightarrow M'}{\lambda x : \tau. M \longrightarrow \lambda x : \tau. M'} \text{ (E-ABS)}$$

$$\frac{}{(\Lambda \alpha :: \kappa. M_1) \langle \tau_2 \rangle \longrightarrow M_1[\tau_2/\alpha]} \text{ (E-TAPPABS)}$$

$$\frac{M_1 \longrightarrow M'_1}{M_1 \langle \tau_2 \rangle \longrightarrow M'_1 \langle \tau_2 \rangle} \text{ (E-TAPP)}$$

$$\frac{M \longrightarrow M'}{\Lambda \alpha :: \kappa. M \longrightarrow \Lambda \alpha :: \kappa. M'} \text{ (E-TABS)}$$

F_ω – Statics – Kinding

$$\begin{array}{c} \frac{}{\Delta, \alpha :: \kappa \vdash \alpha :: \kappa} \quad \frac{}{\Delta \vdash \mathbf{T} :: *} \quad \frac{\Delta \vdash \tau_1 :: * \quad \Delta \vdash \tau_2 :: *}{\Delta \vdash \tau_1 \rightarrow \tau_2 :: *} \\[2ex] \frac{\Delta, \alpha :: \kappa \vdash \tau :: \kappa'}{\Delta \vdash \lambda \alpha :: \kappa. \tau :: \kappa \rightarrow \kappa'} \quad \frac{\Delta \vdash \tau_1 :: \kappa \rightarrow \kappa' \quad \Delta \vdash \tau_2 :: \kappa}{\Delta \vdash \tau_1 \tau_2 :: \kappa'} \\[2ex] \frac{\Delta, \alpha :: \kappa \vdash \tau :: *}{\Delta \vdash \forall \alpha :: \kappa. \tau :: *} \end{array}$$

F_ω – Statics – Type Equivalence

$$\overline{(\lambda \alpha :: \kappa. \tau_1) \tau_2 \equiv \tau_1[\tau_2/\alpha]}$$

$$\frac{\tau_1 \equiv \tau'_1 \quad \tau_2 \equiv \tau'_2}{\tau_1 \rightarrow \tau_2 \equiv \tau'_1 \rightarrow \tau'_2}$$

$$\frac{\tau \equiv \tau'}{\forall \alpha :: \kappa. \tau \equiv \forall \alpha :: \kappa. \tau'}$$

$$\frac{\tau \equiv \tau'}{\lambda \alpha :: \kappa. \tau \equiv \lambda \alpha :: \kappa. \tau'}$$

$$\frac{\tau_1 \equiv \tau'_1 \quad \tau_2 \equiv \tau'_2}{\tau_1 \tau_2 \equiv \tau'_1 \tau'_2}$$

$$\frac{}{\tau \equiv \tau} \quad \frac{\tau \equiv \tau'}{\tau' \equiv \tau}$$

$$\frac{\tau \equiv \tau' \quad \tau \equiv \tau''}{\tau \equiv \tau''}$$

F_ω – Statics – Typing

$$\frac{}{\Delta; \Gamma, X : \tau \vdash X : \tau} \text{ (T-VAR)}$$

$$\frac{\Delta \vdash \tau_1 :: * \quad \Delta; \Gamma, X : \tau_1 \vdash M : \tau_2}{\Delta; \Gamma \vdash (\lambda X : \tau_1. M) : \tau_1 \rightarrow \tau_2} \text{ (T-ABS)}$$

$$\frac{\Delta; \Gamma \vdash M_1 : \tau \rightarrow \tau' \quad \Delta; \Gamma \vdash M_2 : \tau}{\Delta; \Gamma \vdash M_1 M_2 : \tau'} \text{ (T-APP)}$$

$$\frac{\Delta, \alpha :: \kappa; \Gamma \vdash M : \tau}{\Delta; \Gamma \vdash (\lambda \alpha :: \kappa. M) : \forall \alpha :: \kappa. \tau} \text{ (T-TABS)}$$

$$\frac{\Delta; \Gamma \vdash M_1 : \forall \alpha :: \kappa. \tau \quad \Delta \vdash \tau_2 :: \kappa}{\Delta; \Gamma \vdash M_1 \langle \tau_2 \rangle : \tau[\tau_2/\alpha]} \text{ (T-TAPP)}$$

$$\frac{\Delta; \Gamma \vdash M : \tau \quad \tau \equiv \tau' \quad \Delta \vdash \tau' :: *}{\Delta; \Gamma \vdash M : \tau'} \text{ (T-EQ)}$$

F_ω – Properties

Theorem (Preservation)

For all M, M' and τ , if $\bullet; \bullet \vdash M : \tau$ and $M \longrightarrow M'$, then $\bullet; \bullet \vdash M' : \tau$.

Theorem (Progress)

For all M and τ , if $\bullet; \bullet \vdash M : \tau$, then either $M \in \text{Values}$ or $\exists M'. M \longrightarrow M'$.

Strong normalization: Every reduction path starting from a well-typed F_ω term is guaranteed to terminate.

Going Further: Dependent Types

lambda abstraction	$\lambda x : \tau. M$	terms depend on terms
parametric polymorphism	$\Lambda \alpha :: \kappa. M$	terms depend on types
type constructors	$\lambda \alpha :: \kappa. \tau$	types depend on types

Going Further: Dependent Types

lambda abstraction $\lambda x : \tau. M$ terms depend on terms

parametric polymorphism $\Lambda \alpha :: \kappa. M$ terms depend on types

type constructors $\lambda \alpha :: \kappa. \tau$ types depend on types

dependent function types $\Pi x : \tau_1. \tau_2$ types depend on terms

(In the degenerate case, when τ_2 does not mention x , we write $\Pi x : \tau_1. \tau_2$ as $\tau_1 \rightarrow \tau_2$.)

Dependent Function Types – Examples

`FloatList  $n$` represents the type of lists with n float elements.

`nil` : `FloatList 0`

`cons` : $\Pi n : \text{Nat}. \text{Float} \rightarrow \text{FloatList } n \rightarrow \text{FloatList } (\text{succ } n)$

`hd` : $\Pi n : \text{Nat}. \text{FloatList } (\text{succ } n) \rightarrow \text{Float}$

`tl` : $\Pi n : \text{Nat}. \text{FloatList } (\text{succ } n) \rightarrow \text{FloatList } n$

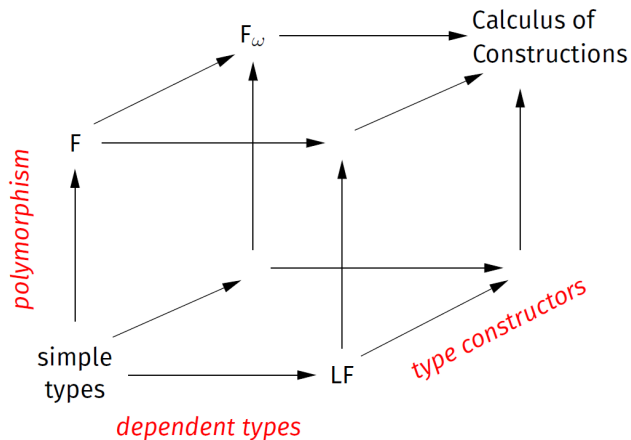
Dependent Function Types – Examples

$\text{consthree} \stackrel{\text{def}}{=} \lambda n : \text{Nat}. \lambda f : \text{Float}. \lambda l : \text{FloatList } n.$
 $\text{cons } (\text{succ } (\text{succ } n)) \ f$
 $\text{cons } (\text{succ } n) \ f$
 $(\text{cons } n \ f \ l)$

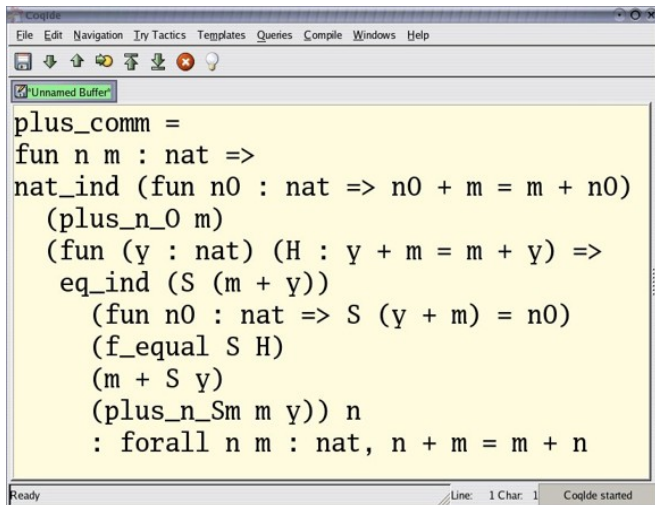
Its type:

$\prod n : \text{Nat}. \text{Float} \rightarrow \text{FloatList } n \rightarrow \text{FloatList } (\text{succ } (\text{succ } (\text{succ } n)))$

Lambda Cube



Coq's Basis: Calculus of Inductive Constructions



The screenshot shows the CoqIDE interface. The menu bar includes File, Edit, Navigation, Try Tactics, Templates, Queries, Compile, Windows, and Help. Below the menu is a toolbar with icons for file operations and execution. The main text area, titled 'Unnamed Buffer', contains the following Coq code:

```
plus_comm =  
fun n m : nat =>  
nat_ind (fun n0 : nat => n0 + m = m + n0)  
  (plus_n_0 m)  
  (fun (y : nat) (H : y + m = m + y) =>  
    eq_ind (S (m + y))  
      (fun n0 : nat => S (y + m) = n0)  
      (f_equal S H)  
      (m + S y)  
      (plus_n_Sm m y)) n  
  : forall n m : nat, n + m = m + n
```

The status bar at the bottom indicates 'Ready', 'Line: 1 Char: 1', and 'CoqIDE started'.

Lambda Cube – Syntax

Unify terms and types:

$$A ::= x \mid s \mid A A \mid \lambda x : A. A \mid \Pi x : A. A$$

Here the sort s is either $*$, denoting the type of types, or $\square$, which is one level up, denoting the type of kinds (such as $*$ and $\Pi x : *. *$).

$$s ::= * \mid \square$$

Lambda Cube – Syntax

Unify terms and types:

$$A ::= x \mid s \mid A A \mid \lambda x : A. A \mid \Pi x : A. A$$

Here the sort s is either $*$, denoting the type of types, or $\square$, which is one level up, denoting the type of kinds (such as $*$ and $\Pi x : *. *$).

$$s ::= * \mid \square$$

Typing context:

$$\Gamma ::= \bullet \mid \Gamma, x : A$$

Lambda Cube – Typing Rules

$$\frac{}{\bullet \vdash * : \square} \text{ (SORT)} \qquad \frac{\Gamma \vdash A : s \quad x \notin \Gamma}{\Gamma, x : A \vdash x : A} \text{ (VAR)}$$

$$\frac{\Gamma \vdash A : B \quad \Gamma \vdash C : s \quad x \notin \Gamma}{\Gamma, x : C \vdash A : B} \text{ (WEAK)}$$

$$\frac{\Gamma \vdash A : s_1 \quad \Gamma, x : A \vdash B : s_2}{\Gamma \vdash \Pi x : A. B : s_2} \text{ (FORM)}$$

$$\frac{\Gamma \vdash M : \Pi x : A. B \quad \Gamma \vdash N : A}{\Gamma \vdash MN : B[A/x]} \text{ (APP)}$$

$$\frac{\Gamma, x : A \vdash M : B \quad \Gamma \vdash \Pi x : A. B : s}{\Gamma \vdash \lambda x : A. M : \Pi x : A. B} \text{ (ABS)}$$

$$\frac{\Gamma \vdash A : B \quad \Gamma \vdash B' : s \quad B =_{\beta} B'}{\Gamma \vdash A : B'} \text{ (CONV)}$$

Lambda Cube – Expressing Different Dependencies

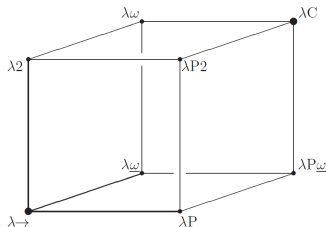
$$\frac{\Gamma \vdash A : s_1 \quad \Gamma, x : A \vdash B : s_2}{\Gamma \vdash \lambda x : A. B : s_2} \text{ (FORM)}$$

$x : A : s_1$	$b : B : s_2$	(s_1, s_2)	$\lambda x : A. b$	from
*	*	$(*, *)$	term depending on term	STLC
$\square$	*	$(\square, *)$	term depending on type	System F
$\square$	$\square$	$(\square, \square)$	type depending on type	$\lambda\omega$
*	$\square$	$(*, \square)$	type depending on term	LF

Lambda Cube

system: combinations (s_1, s_2) allowed:

$\lambda \rightarrow$	$(*, *)$		
$\lambda 2$	$(*, *)$	$(\square, *)$	
$\lambda \underline{\omega}$	$(*, *)$	$(\square, \square)$	
$\lambda \underline{P}$	$(*, *)$		$(*, \square)$
$\lambda \omega$	$(*, *)$	$(\square, *)$	$(\square, \square)$
$\lambda P2$	$(*, *)$	$(\square, *)$	$(*, \square)$
$\lambda P \underline{\omega}$	$(*, *)$	$(\square, \square)$	$(*, \square)$
$\lambda P \omega = \lambda C$	$(*, *)$	$(\square, *)$	$(\square, \square)$



Taken from the book *Type Theory and Formal Proof: An Introduction* by Rob Nederpelt and Herman Geuvers.